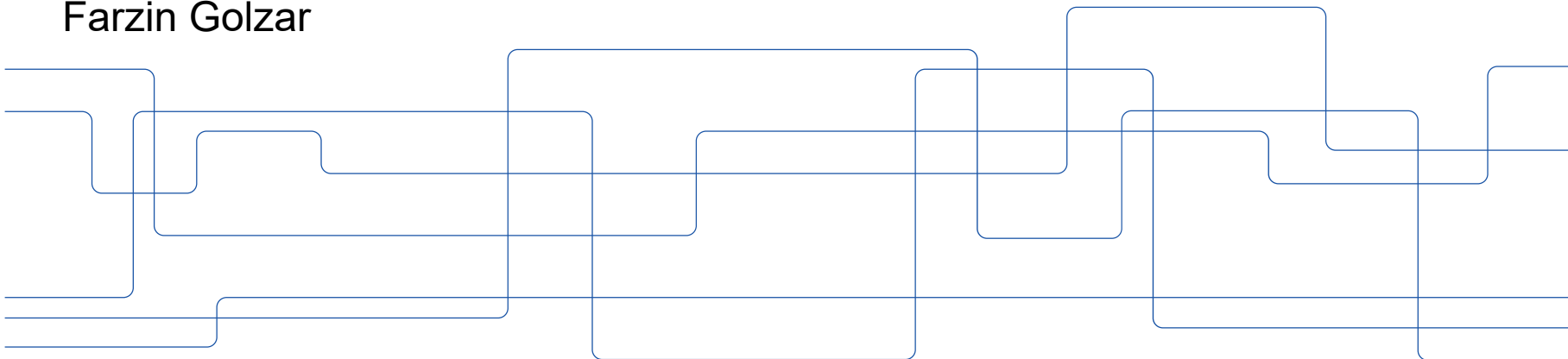


Shallow Neural Networks (hidden units and hidden layers)

Farzin Golzar



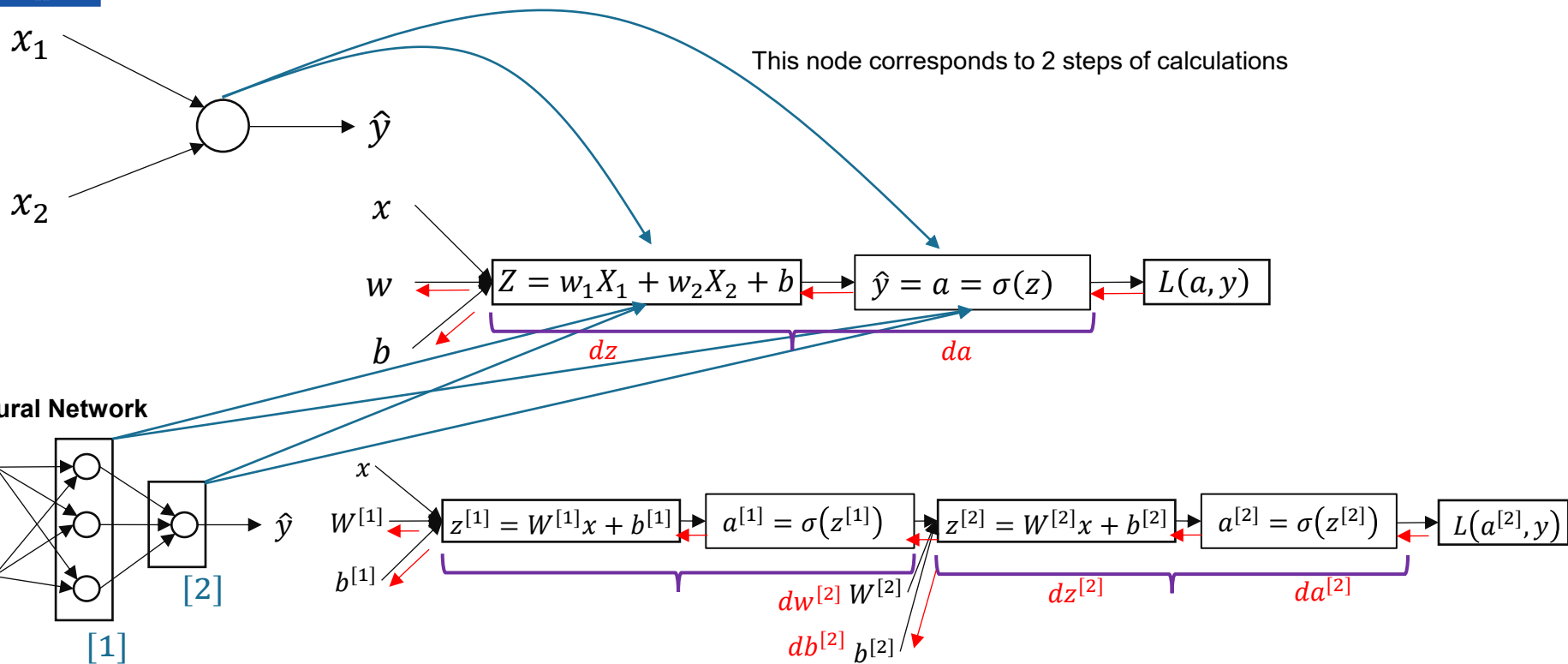


Learning Objectives

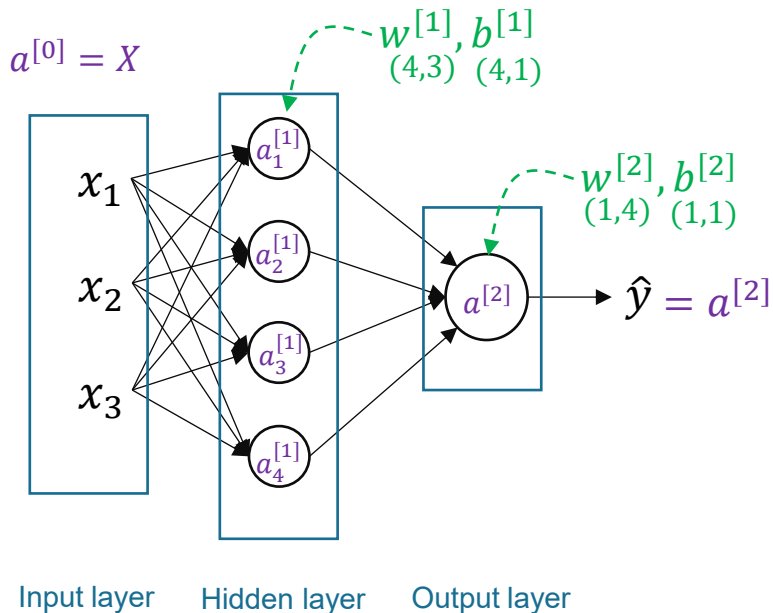
- Developing neural network on top of logistic regression
- Describe hidden units and hidden layers
- Use units with a non-linear activation function, such as tanh
- Implement forward and backward propagation
- Apply random initialization to your neural network
- Increase fluency in Deep Learning notations and Neural Network Representations

Neural Networks Overview

[1]: layer number
(1): training example number



Neural Network Representation



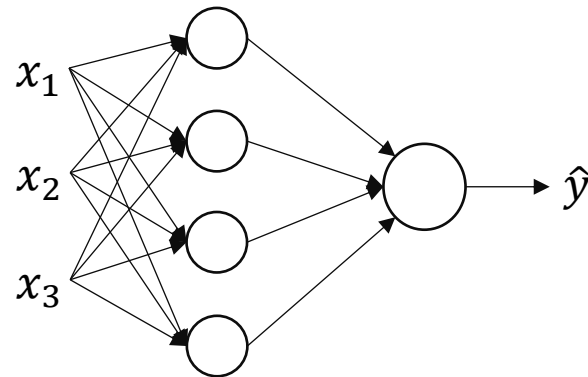
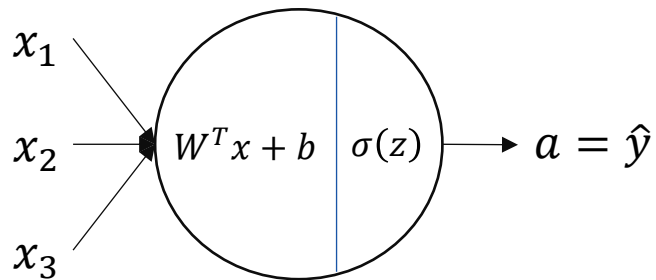
$$a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ \vdots \\ a_4^{[1]} \end{bmatrix}$$

This is a **2 layers NN**, because the input layer is not count.

Why hidden layer: The term hidden layer refers to the fact that in the training set, the true values for these nodes in the middle are not observed.

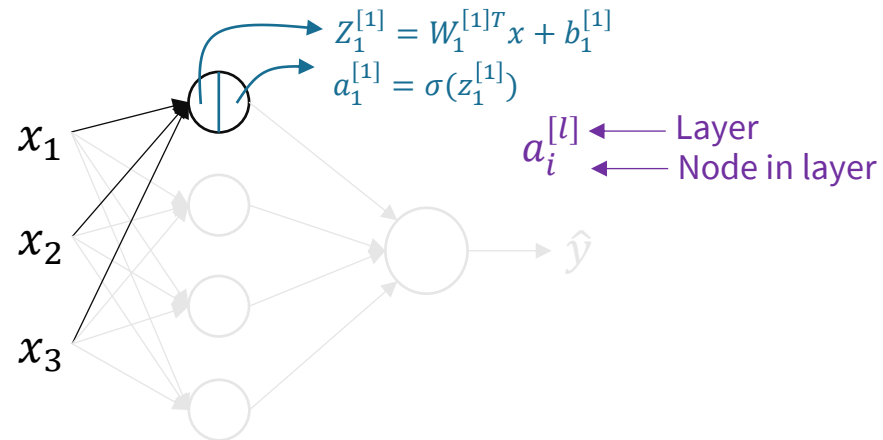
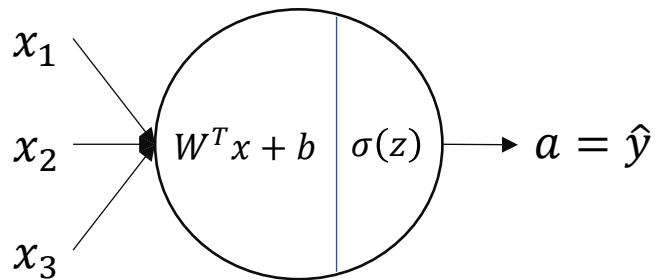
You see what the inputs are. You see what the output should be. But the things in the hidden layer are not seen in the training set.

Computing a Neural Network's Output

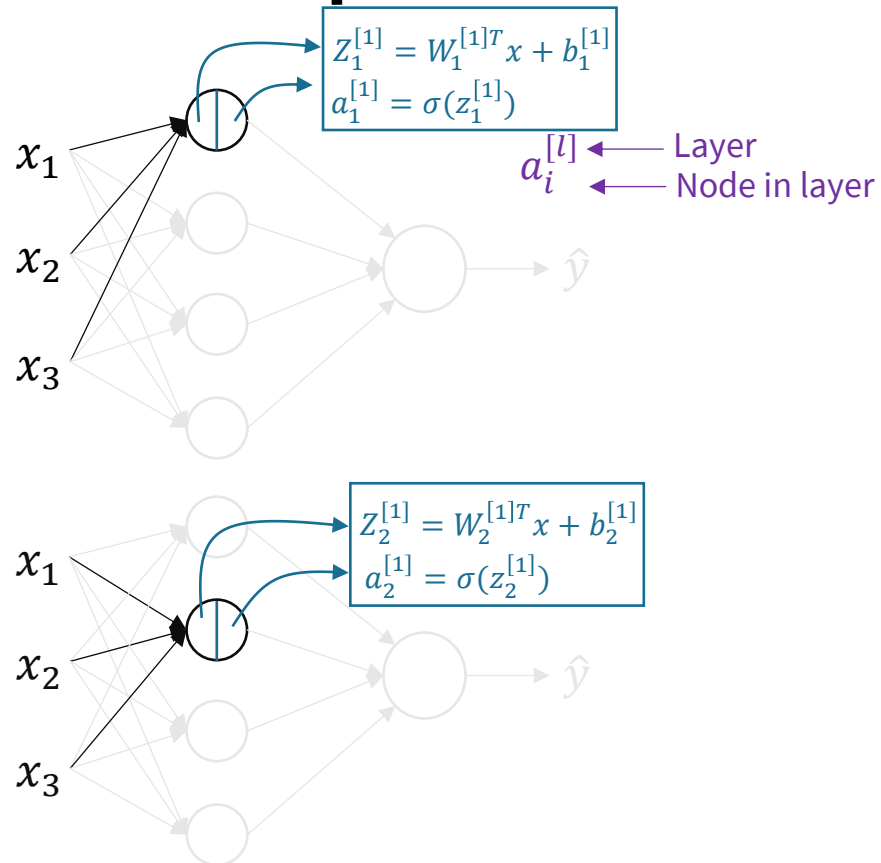
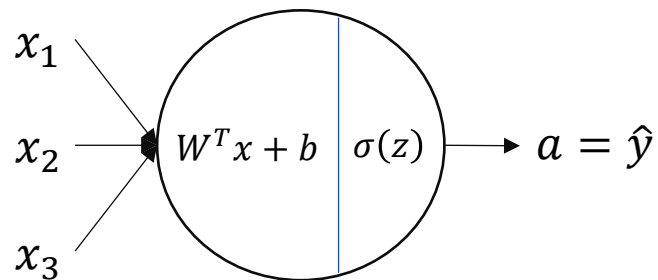


→
A neural network just does this a lot more times

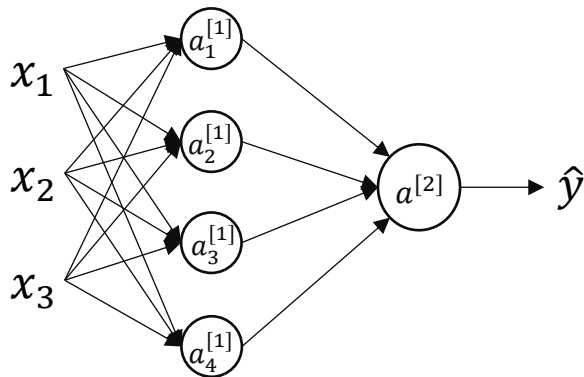
Computing a Neural Network's Output



Computing a Neural Network's Output



Computing a Neural Network's Output

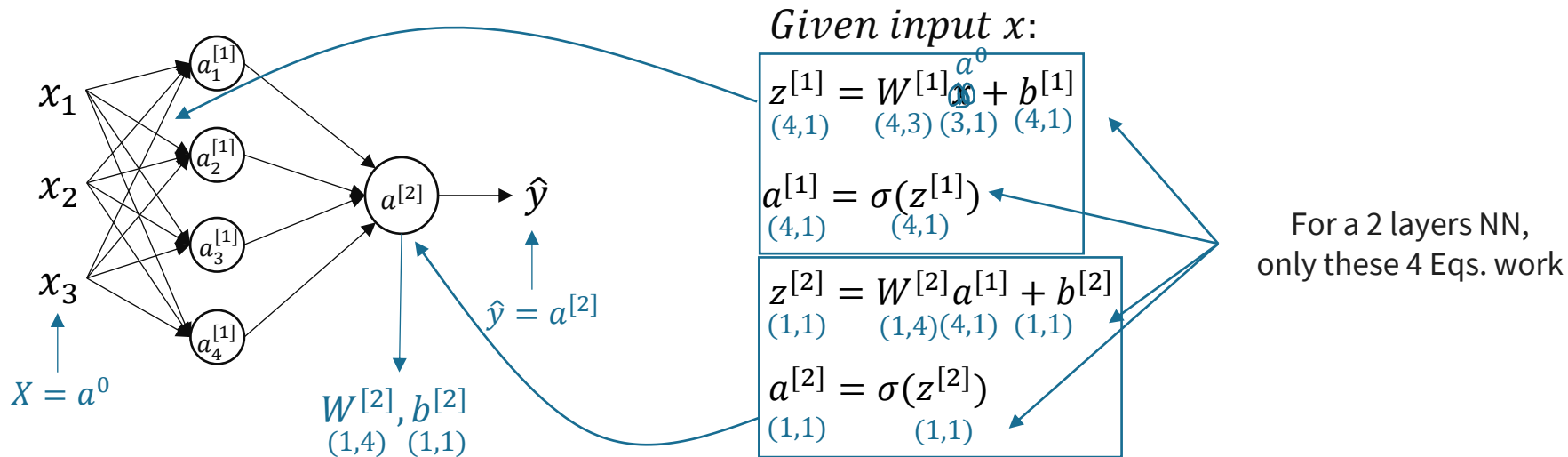


$$\left. \begin{aligned} z_1^{[1]} &= w_1^{[1]T} x + b_1^{[1]}, & a_1^{[1]} &= \sigma(z_1^{[1]}) \\ z_2^{[1]} &= w_2^{[1]T} x + b_2^{[1]}, & a_2^{[1]} &= \sigma(z_2^{[1]}) \\ z_3^{[1]} &= w_3^{[1]T} x + b_3^{[1]}, & a_3^{[1]} &= \sigma(z_3^{[1]}) \\ z_4^{[1]} &= w_4^{[1]T} x + b_4^{[1]}, & a_4^{[1]} &= \sigma(z_4^{[1]}) \end{aligned} \right\} \begin{array}{l} \text{No for loop} \\ \text{Vectorization} \end{array}$$

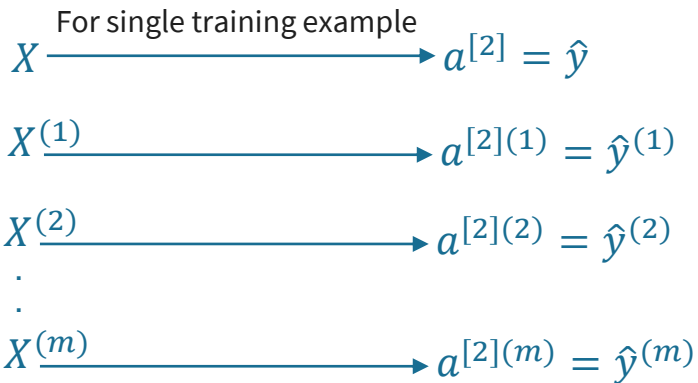
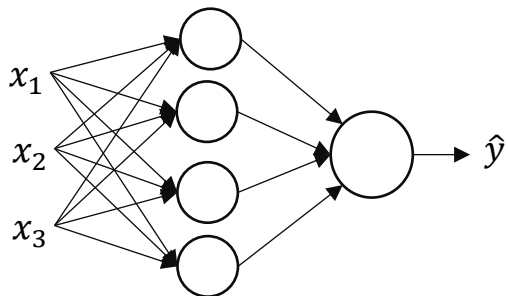
$$\begin{aligned} Z^{[1]} &= \begin{bmatrix} - & -w_1^{[1]T} & - \\ - & -w_2^{[1]T} & - \\ - & -w_3^{[1]T} & - \\ - & -w_4^{[1]T} & - \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix} = \begin{bmatrix} W_1^{[1]T} X + b_1^{[1]} \\ W_2^{[1]T} X + b_2^{[1]} \\ W_3^{[1]T} X + b_3^{[1]} \\ W_4^{[1]T} X + b_4^{[1]} \end{bmatrix} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} \\ &\quad (4,3) \qquad (4,1) \end{aligned}$$

$$a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} = \sigma(Z^{[1]})$$

Computing a Neural Network's Output



Vectorization across multiple examples



$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

for $i = 1$ to m :

$$z^{[1]}(i) = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$z^{[2]}(i) = W^{[2]}a^{[1]}(i) + b^{[2]}$$

$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$

We should get
rid of this
for loop

Vectorization across multiple examples

for $i = 1$ to m :

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

$$X = \begin{bmatrix} | & | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & | & | \end{bmatrix} \begin{matrix} \updownarrow \\ n_x \\ \updownarrow \end{matrix}$$

$\xleftarrow{m}$

We went from lower case vector x to just capital case X matrix by **stacking** up the lower case x s in different columns.

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

You now know how to implement in your network with **vectorization**.

$$Z^{[1]} = \begin{bmatrix} | & | & | & | \\ z^{1} & z^{[1](2)} & \dots & z^{[1](m)} \\ | & | & | & | \end{bmatrix}$$

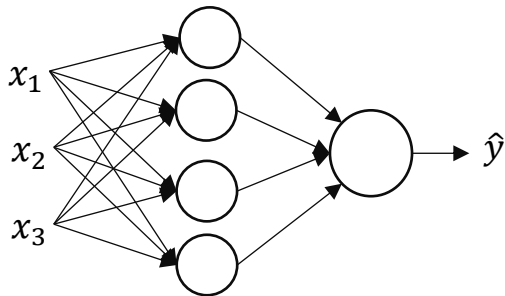
$$A^{[1]} = \begin{bmatrix} | & | & | & | \\ a^{1} & a^{[1](2)} & \dots & a^{[1](m)} \\ | & | & | & | \end{bmatrix}$$

$\xleftarrow[m]{\text{training examples}}$

$\updownarrow$ hidden unit number

The same for $Z^{[2]}$ and $A^{[2]}$

Recap of vectorization across multiple examples



$$X = \begin{bmatrix} | & | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & | & | \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} | & | & | & | \\ a^{1} & a^{[1](2)} & \dots & a^{[1](m)} \\ | & | & | & | \end{bmatrix}$$

for $i = 1$ to m :

$$z^{[1]}(i) = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$z^{[2]}(i) = W^{[2]}a^{[1]}(i) + b^{[2]}$$

$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$

Vectorized by

$$Z^{[1]} = W^{[1]} \overset{A^{[0]}}{X} + b^{[1]} \rightarrow Z^{[1]} = W^{[1]}A^{[0]} + b^{[1]}$$

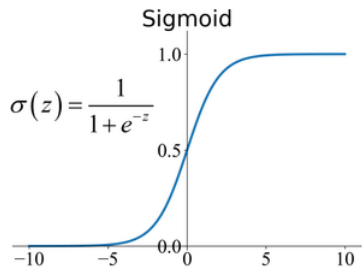
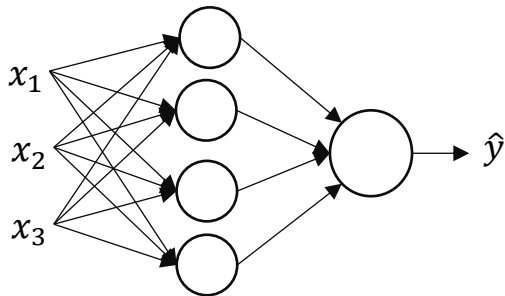
$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

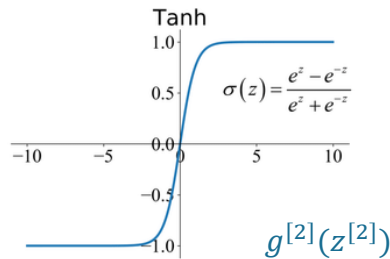
This shows that the different layers of a neural network are roughly doing the same thing or just doing the same computation over and over.

Activation functions



For output layer, Sigmoid outperforms Tanh, because the output range is 0 and 1.

$$g^{[1]}(z^{[1]})$$



Tanh often trains better than sigmoid in hidden layers because tanh is zero-centered, whereas sigmoid outputs are always positive.

$$g^{[2]}(z^{[2]})$$

One of the **downsides** of both the sigmoid function and the tanh function is that if z is either very large or very small, then the gradient of the derivative of the slope of this function becomes very small.

Given x :

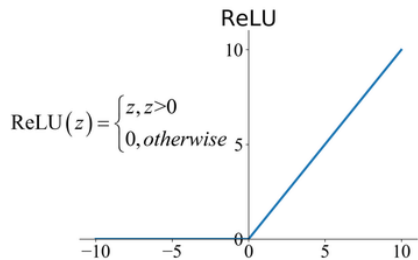
$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$\rightarrow a^{[1]} = \cancel{\sigma(z^{[1]})} g(z^{[1]})$$

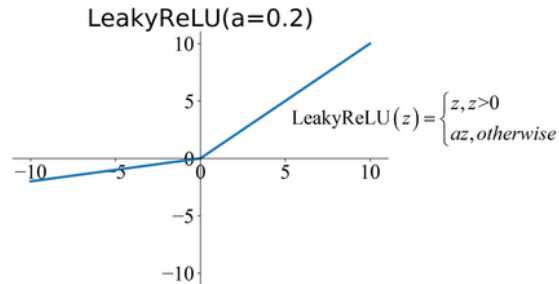
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$\rightarrow a^{[2]} = \cancel{\sigma(z^{[2]})} g(z^{[2]})$$

We have been using Sigmoid function as the **activation function**



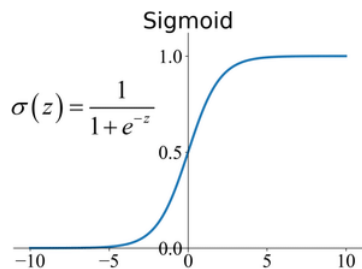
One **disadvantage** of the ReLU is that the derivative is equal to zero when z is negative.



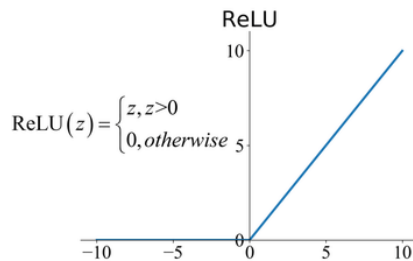
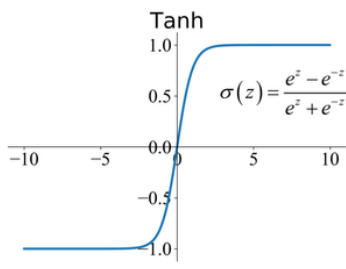
This usually works better than the value activation function. Although, it's just not used as much in practice.

Pros and cons of activation layers

- The **advantage** of both the ReLU and the Leaky ReLU is that for a lot of the space of z , the derivative of the activation function, the slope of the activation function is very **different from zero**.
- In practice, using the ReLU activation function, your neural network will often **learn much faster** than when using the tanh or the sigmoid activation function.

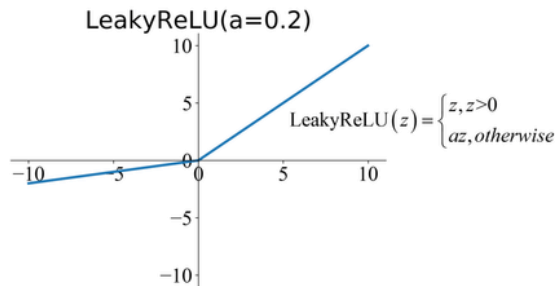


Do not use this except
for the **output layer**.



The default, the **most commonly used**
activation function is
the ReLU

Not sure what else
to use, use this one

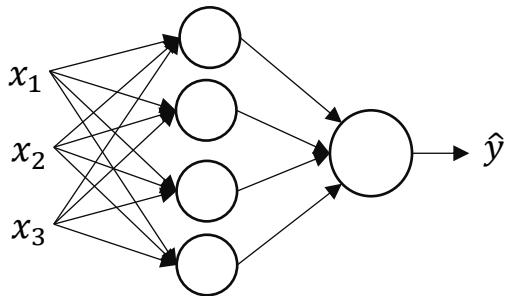


Feel free also to try the
Leaky ReLU

It is sometimes difficult to get good guidelines for exactly what will work best for your problem.

If you're not sure which one of these activation functions work best, **try them all**.

Why do you need Non-Linear Activation Functions?



Given x :

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = \cancel{\sigma(z^{[1]})} z^{[1]}$$

$$g(z) = z$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \quad \text{Linear activation function}$$

$$a^{[2]} = \cancel{\sigma(z^{[2]})} z^{[2]}$$

$$a^{[1]} = z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[2]} = z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

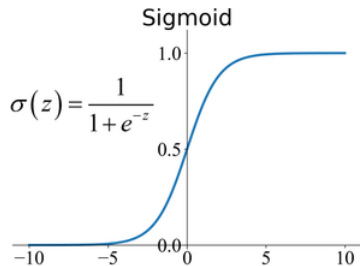
$$a^{[2]} = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}$$

$$= \underbrace{(W^{[2]}W^{[1]})}_{\hat{W}}x + \underbrace{(W^{[2]}b^{[1]} + b^{[2]})}_{\hat{b}}$$

$$= \hat{W}x + \hat{b}$$

- The neural network is just outputting a linear function of the input.
- If you use a linear activation function, then no matter how many layers your neural network has, all it's doing is just computing a linear activation function.

Derivatives of activation function



$$\dot{g}(z) = \frac{d}{dz} g(z) = \text{slope of } g(x) \text{ at } z:$$

$$a = g(z)$$

$$= \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}}\right)$$

$$= g(z)(1 - g(z)) = a \times (1 - a)$$

$$\dot{g}(z) = a \times (1 - a)$$

The advantage of this formula is that if you've already computed the value for a , then by using this expression, you can very quickly compute the value for the $\dot{g}$ as well.

$$Z = 10 \rightarrow g(z) \approx 1 \rightarrow \frac{d}{dz} g(z) \approx 1 \times (1 - 1) \approx 0$$

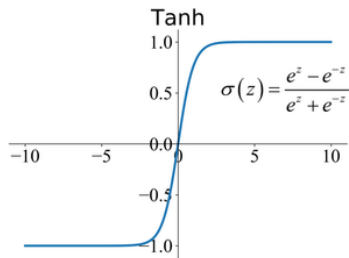
$$Z = -10 \rightarrow g(z) \approx 0 \rightarrow \frac{d}{dz} g(z) \approx 0 \times (1 - 0) \approx 0$$

$$Z = 0 \rightarrow g(z) = \frac{1}{2} \rightarrow \frac{d}{dz} g(z) = \frac{1}{2} \times \left(1 - \frac{1}{2}\right) = \frac{1}{4}$$

Derivatives of activation function

$$\dot{g}(z) = \frac{d}{dz} g(z) = \text{slope of } g(x) \text{ at } z:$$

$$= 1 - (\tanh(z))^2$$



$$Z = 10 \rightarrow g(z) \approx 1 \rightarrow \frac{d}{dz} g(z) \approx (1 - 1) \approx 0$$

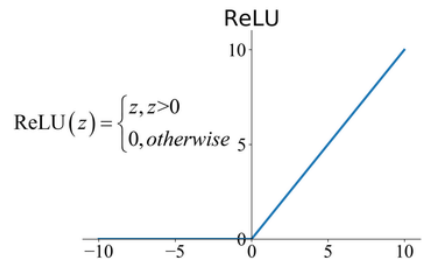
$$Z = -10 \rightarrow g(z) \approx -1 \rightarrow \frac{d}{dz} g(z) \approx (1 - (-1)^2) \approx 0$$

$$Z = 0 \rightarrow g(z) = 0 \rightarrow \frac{d}{dz} g(z) = (1 - 0) = 1$$

$$a = g(z)$$

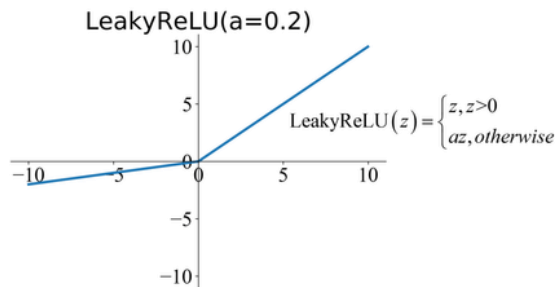
$$\dot{g}(z) = 1 - a^2$$

Derivatives of activation function



$$\dot{g}(z) = \frac{d}{dz} g(z) = \text{slope of } g(x) \text{ at } z:$$

$$= \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$



$$\dot{g}(z) = \frac{d}{dz} g(z) = \text{slope of } g(x) \text{ at } z:$$

$$= \begin{cases} 0.2 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

Gradient descent for neural network

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$
 $(n^{[1]}, n^{[0]}) \quad (n^{[1]}, 1) \quad (n^{[2]}, n^{[1]}) \quad (n^{[2]}, 1)$

$$n_x = n^{[0]}, n^{[1]}, n^{[2]} = 1$$

Cost function: $J(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}) = \frac{1}{m} \sum_{i=1}^m L(\hat{y} - y)$

$\swarrow a^{[2]}$

Gradient descent:

Repeat {

Compute predictions $(\hat{y}^{(i)}, i = 1, \dots, m)$

$$dW^{[1]} = \frac{dJ}{dW^{[1]}}, db^{[1]} = \frac{dJ}{db^{[1]}}, dW^{[2]} = \frac{dJ}{dW^{[2]}}, db^{[2]} = \frac{dJ}{db^{[2]}}$$

$$W^{[1]} = W^{[1]} - \alpha dW^{[1]},$$

$$b^{[1]} = b^{[1]} - \alpha db^{[1]},$$

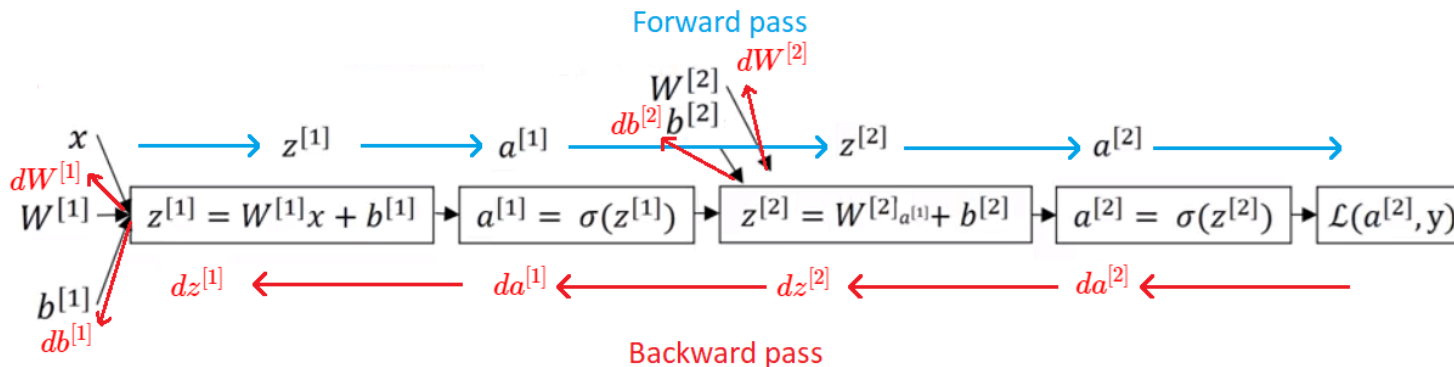
$$W^{[2]} = W^{[2]} - \alpha dW^{[2]},$$

$$b^{[2]} = b^{[2]} - \alpha db^{[2]}$$

}

The key is to calculate the **derivatives**

Derivatives: computational graph



The same approach as logistic regression, but here we have 2 layers



Formulas for computing derivatives

Forward propagation:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = g^{[1]}(z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(z^{[2]}) = \sigma(Z^{[2]})$$

Back propagation:

$$dZ^{[2]} = A^{[2]} - Y \quad Y = [y^{(1)}, y^{(2)}, \dots, y^{(m)},]$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

Horizontally Prevents rank 1 array (n,)

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} \times g^{[1]}(Z^{[1]})$$

Element-wise product

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

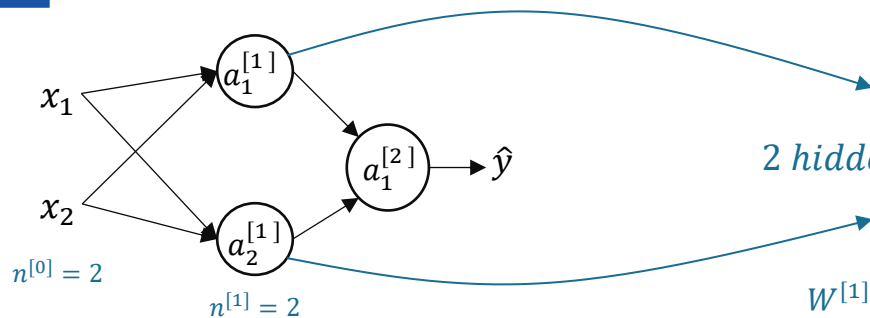
$$db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True})$$



Initialization

- For **logistic regression**, it was okay to initialize the weights to zero.
- For a **neural network** of initialize the weights to parameters to all zero and then applied gradient descent, **it won't work**.
- Why?

Random Initialization



2 hidden units will be completely identical or symmetric

$$W^{[1]} = W^{[1]} - \alpha dW^{[1]}$$

Assume:

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \quad b^{[1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

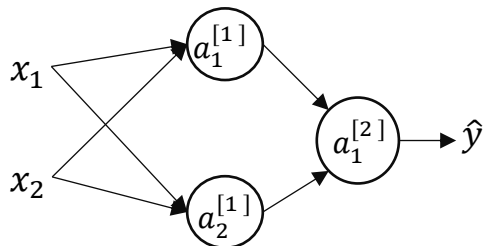
$$a_1^{[1]} = a_2^{[1]} \quad dz_1^{[1]} = dz_2^{[1]}$$

$$W^{[1]} = \begin{bmatrix} u & u \\ v & v \end{bmatrix} \quad \text{There is no point to have more than 1 hidden unit}$$

For bigger neural network with more hidden layer, the same thing will happen.

*The solution is to initialize the parameters **randomly**.*

Random Initialization



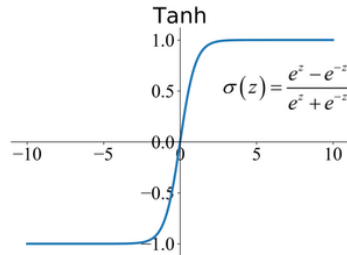
Where does it come from? Why is it so small?

$$W^{[1]} = np.random.rand((2,2)) \times 0.01$$

$$b^{[1]} = np.zeros((2,1))$$

$$W^{[2]} = np.random.rand((1,2)) \times 0.01$$

$$b^{[2]} = np.zeros((1,1))$$



If the W is too large, z will be either very large or very small. And so in that case, you're more likely to end up at the fat parts of the tanh function or the sigmoid function, where the slope or the gradient is very small. Meaning that gradient descent will be very slow. So learning was very slow.

Formulas for computing derivatives

Forward propagation:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = g^{[1]}(z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(z^{[2]}) = \sigma(Z^{[2]})$$

Back propagation:

$$dZ^{[2]} = A^{[2]} - Y \quad Y = [y^{(1)}, y^{(2)}, \dots, y^{(m)},]$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} \times g^{[1]'}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True})$$



What to do next

- Go to CANVAS
- Solve HW2



References

- [Speech and Language Processing](#). Daniel Jurafsky & James H. Martin. Draft of January 7, 2023.
- [Deeplearning.ai](#)
- [Implementing-a-neural-network-from-scratch/](#)